


# NestJS Assignment 2

## 🎯 Objective

The goal of this assignment is to build a Users Module to practice NestJS fundamentals such as Modules, Controllers, Services, DTOs, Query Params, and Pipes. By the end of this task, you should be able to:

- Organize code into Modules.
- Create and use Controllers & Services.
- Validate input with DTOs & Pipes.
- Handle query parameters in routes.

## 📌 Task Steps

### 1️⃣ Create Users Module

To begin, you need to generate the Users module along with its controller and service. Use the Nest CLI for this purpose:

nest g module users
nest g controller users
nest g service users

This will create the necessary files and structure for your Users module.

### 2️⃣ Implement Routes

Next, you will implement the following routes in the UsersController:

- **GET /users**: This route should return a list of users.
- **POST /users**: This route should create a new user.

### 3️⃣ Create DTO with Validation

Create a Data Transfer Object (DTO) to validate the data for creating a new user. This will typically be defined in a file named create-user.dto.ts. Use the class-validator package to enforce validation rules for incoming data.

### 4️⃣ Add Query Param Filtering

Enhance the GET /users route by adding query parameter filtering. Specifically, handle requests like **GET /users?role=admin** to return only users with the role admin.

### 5️⃣ Add Pipes for Validation & Transformation

Enable the ValidationPipe globally in your main.ts file to ensure incoming requests are validated against your DTOs.

## 📝 Deliverables

- **Screenshot** showing the output of **GET /users**.
- **Screenshot** demonstrating success and failure scenarios for **POST /users** with validation.
- **Screenshot** of the filtered output for **GET /users?role=admin**.

## ✅ Evaluation Criteria

Your submission will be evaluated based on the following criteria:

- **Correct module structure**: Ensure that UsersModule, UsersController, and UsersService are correctly implemented.
- **Working routes**: Both GET and POST routes should work as expected, including handling query parameters.
- **DTO validation**: Use class-validator for proper data validation.
- **Code quality**: Ensure proper naming conventions, formatting, and comments where necessary.

## 📤 Submission Guidelines

1. Push your project to a GitHub repository named nestjs-fundamentals.
2. Optionally, deploy your backend using platforms like Render, Railway, or Netlify Functions.
3. Submit the following:
    - GitHub Repo Link
    - The Deploy Link in README.MD file

Following these steps will ensure you successfully complete the assignment and demonstrate your understanding of NestJS fundamentals. Good luck!